

ShivaAI Jarvis — Developer Quick Start Guide

For: Engineering team starting development

Duration to read: 10 minutes

Next step: Setup your dev environment

PROJECT AT A GLANCE

What	Details
Project	ShivaAI Jarvis v5+ (Autonomous AI Operating System)
Duration	28 days to production launch
Team	6-12 engineers
Goal	Launch enterprise-ready personal AI assistant with trading, code, teaching, predictions, voice
Key Tech	FastAPI, React, LangGraph, Qdrant, Neo4j, Kubernetes

WHERE TO FIND EVERYTHING

Essential Documents

/docs/	
— SHIVAAI_1MONTH_ENTERPRISE_ROADMAP.md	← Development timeline (28 days)
— SHIVAAI_PROJECT_DOCUMENTATION.md	← Complete specifications
— DEVELOPER_QUICK_START.md	← You are here
— API_REFERENCE.md	← API endpoints (generate)
— DATABASE_SCHEMA.md	← SQL schema (generate)
— ARCHITECTURE_DIAGRAMS.md	← System diagrams (generate)
— SECURITY_CHECKLIST.md	← Security requirements
/src/	
— backend/	← Python FastAPI code
— frontend/	← React TypeScript code
— services/	← Specialized services
— infra/	← Kubernetes configs
— scripts/	← Automation scripts

Quick Links

- **Roadmap:** [1-Month Timeline](#)
 - **Full Specs:** [Project Documentation](#)
 - **Architecture:** [System Design](#)
 - **APIs:** [API Specifications](#)
 - **Database:** [PostgreSQL Schema](#)
 - **Security:** [Security & Compliance](#)
-

LOCAL SETUP (30 minutes)

Prerequisites

```
bash

# Required
- Python 3.12+
- Node.js 18+
- Docker & Docker Compose
- Git
- PostgreSQL 15 (or via Docker)
- Redis 7+ (or via Docker)

# Verify installations
python --version      # 3.12+
node --version        # 18+
docker --version      # 20+
git --version         # 2.40+
```

1. Clone Repository

```
bash

git clone https://github.com/your-org/shivaai-jarvis.git
cd shivaai-jarvis

# Create main branches
git checkout -b develop origin/develop
```

2. Backend Setup

```
bash

# Create Python virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt # Dev tools: pytest, black, pylint

# Create .env file
cp .env.example .env

# Edit .env
nano .env

# Set:
# DATABASE_URL=postgresql://user:password@localhost:5432/shivaai
# REDIS_URL=redis://localhost:6379
# OPENAI_API_KEY=sk-...
# ANTHROPIC_API_KEY=sk-...
# JWT_SECRET=your-secret-key

# Initialize database
alembic upgrade head

# Start backend
uvicorn src.main:app --reload --port 8000
# Server running at http://localhost:8000
```

3. Frontend Setup

```
bash
```

```
cd apps/web

# Install dependencies
npm install

# Create .env.local
cp .env.example .env.local
# Set:
# REACT_APP_API_URL=http://localhost:8000
# REACT_APP_WS_URL=ws://localhost:8000

# Start dev server
npm run dev
# App running at http://localhost:5173
```

4. Docker Setup (Full Stack)

```
bash

# From project root
docker-compose up -d

# Services will be available:
# - Frontend: http://localhost:3000
# - Backend: http://localhost:8000
# - PostgreSQL: localhost:5432
# - Redis: localhost:6379
# - Qdrant: http://localhost:6333
# - Neo4j: http://localhost:7474

# Check logs
docker-compose logs -f backend

# Stop all services
docker-compose down
```

5. Verify Setup

```
bash
```

```
# Test backend
curl http://localhost:8000/health

# Test frontend loads
curl http://localhost:3000

# Test database
psql postgresql://user:password@localhost:5432/shivaai -c "\dt"

# Run tests
pytest tests/unit -v
npm test
```

TEAM ROLES & QUICK ASSIGNMENTS

Week 1 Assignments

Backend Lead + 1 Engineer:

- ☐ Day 1-2: Database schema, migrations
- ☐ Day 2-3: FastAPI auth, gateway
- ☐ Day 4-5: AI runtime abstraction
- ☐ Day 5-6: Basic chat endpoints

Frontend Lead + 1 Engineer:

- ☐ Day 1-2: React + Zustand setup
- ☐ Day 2-3: Auth UI + routing
- ☐ Day 4-5: Chat UI skeleton
- ☐ Day 5-6: Streaming responses

DevOps/Security:

- ☐ Day 1-2: Docker, docker-compose
- ☐ Day 2-3: CI/CD pipeline (GitHub Actions)
- ☐ Day 4-5: Security middleware

AI/ML:

- ☐ Day 2-3: LLM provider adapters
 - ☐ Day 4-5: Embedding pipeline (for Week 2)
-

ARCHITECTURE OVERVIEW (2-min version)

```
graph TD
    User[User (Chat/Voice/API)] --> Gateway[FastAPI Gateway]
    Gateway --> Orchestrator[LangGraph Orchestrator]
    Orchestrator --> Memory[Memory + Knowledge]
    Memory --> Modules[Specialized Modules]
    Modules --> Response[Response Stream]
    Response --> UserResult[User sees result]
```

User (Chat/Voice/API)

↓

[FastAPI Gateway]

- Auth (OAuth2/JWT)
- Rate limiting
- Routing

↓

[LangGraph Orchestrator]

- Planner agent
- Research agent
- Executor agent
- Reflection agent

↓

[Memory + Knowledge]

- Qdrant (vectors)
- Neo4j (relationships)
- PostgreSQL (primary)

↓

[Specialized Modules]

- RAG, Voice, Workflow
- Trading, Code, Teaching
- Predictions

↓

[Response Stream]

(SSE/WebSocket)

↓

User sees result

Key Flow:

1. Request → Gateway (auth, validate)
2. Planner agent breaks down task
3. Specialist agents execute (with tools)
4. Reflection agent validates
5. Memory agent stores
6. Response streams back

COMMON COMMANDS

Git Workflow

```
bash

# Create feature branch
git checkout -b feat/chat-streaming

# Make changes
git add .
git commit -m "feat: add streaming chat responses"

# Push and create PR
git push origin feat/chat-streaming
# Open PR on GitHub

# After review, merge via GitHub UI
```

Backend Development

```
bash

# Run server with auto-reload
uvicorn src.main:app --reload

# Run specific test
pytest tests/unit/test_chat.py::test_streaming -v

# Run all tests with coverage
pytest --cov=src

# Format code
black src/

# Lint code
pylint src/

# Type check
mypy src/
```

Frontend Development

```
bash
```

```
# Run dev server
npm run dev

# Run tests
npm test

# Build for production
npm run build

# Format code
npm run format

# Lint code
npm run lint
```

Docker

```
bash

# Build image
docker build -t shivaai-jarvis:latest .

# Run container
docker run -p 8000:8000 shivaai-jarvis:latest

# View logs
docker logs -f container_id

# Stop container
docker stop container_id
```

Database

```
bash
```

```
# Create migration
alembic revision --autogenerate -m "add user_feedback table"

# Apply migrations
alembic upgrade head

# Rollback one migration
alembic downgrade -1

# Access database
psql postgresql://user:password@localhost:5432/shivaai
```

DAILY STANDUP TEMPLATE

Every morning, post in Slack:

```
:wave: [Your Name]
```

```
What I did yesterday:
```

- Implemented streaming chat endpoint
- Added 15 unit tests

```
What I'm doing today:
```

- Add artifact generation
- Review PR #42

```
Blockers:
```

- Waiting for OpenAI API key (shared with tech lead)

```
Help needed:
```

- How to structure LangGraph state? (ping @backend-lead)

DEBUGGING TIPS

Backend Issues

```
bash
```

```
# Check if server is running
curl http://localhost:8000/health
# Expected: {"status": "healthy"}

# View logs
docker-compose logs backend

# Check database connection
python -c "from src.db import SessionLocal; print(SessionLocal())"

# Check Redis connection
redis-cli ping
# Expected: PONG

# Test LLM provider
python -c "from src.ai.runtime import LLMRouter; LLMRouter().test_openai()"
```

Frontend Issues

```
bash

# Check API connection
curl http://localhost:8000/api/v1/health

# View network requests
# Open Chrome DevTools → Network tab

# Check local storage
# Open Chrome DevTools → Application → Local Storage

# Check browser console for errors
# Open Chrome DevTools → Console
```

Database Issues

```
bash
```

```
# Verify connection
psql postgresql://user:password@localhost:5432/shivaai -c "SELECT 1"

# List all tables
\dt

# Check migration status
alembic current

# View query performance
EXPLAIN ANALYZE SELECT * FROM conversations;
```

LEARNING RESOURCES

For This Project

- **FastAPI:** <https://fastapi.tiangolo.com/>
- **LangGraph:** <https://langchain-ai.github.io/langgraph/>
- **React:** <https://react.dev/>
- **Zustand:** <https://github.com/pmndrs/zustand>
- **SQLAlchemy:** <https://sqlalchemy.org/>

For Concepts

- **Multi-agent systems:** <https://arxiv.org/abs/2308.12966>
 - **RAG:** <https://arxiv.org/abs/2005.11401>
 - **Fine-tuning:** <https://platform.openai.com/docs/guides/fine-tuning>
 - **Voice interfaces:** <https://openai.com/research/whisper>
-

CRITICAL PATHS (Don't Miss These)

Must Ship by Day 28

- ✓ Basic chat (Week 1)
- ✓ Multi-agent orchestration (Week 2)
- ✓ Voice interface (Week 2)
- ✓ Memory system (Week 2)
- ✓ Trading AI (Week 3)

- ✓ Code assistant (Week 3)
- ✓ Security hardening (Weeks 1-4)
- ✓ Deployment pipeline (Week 4)

High Risk

- ⚠ **LLM latency** → Test early, have fallback models
 - ⚠ **Voice lag** → WebRTC optimization critical
 - ⚠ **Database scaling** → Load test by Day 15
 - ⚠ **Fine-tuning feedback** → Start collecting by Day 14
-

GET HELP

Who to Ask

Question	Ask
How do I...?	#help Slack channel
Architecture decision	@tech-lead
API question	@backend-lead
UI/Frontend	@frontend-lead
Infrastructure	@devops
Security concern	@security-lead

How to Report Issues

1. Check documentation first
2. Search GitHub issues
3. Ask in #help channel
4. Create GitHub issue with:
 - Clear title
 - What you tried
 - Expected vs actual result
 - Environment (OS, versions)

✅ LAUNCH CHECKLIST (Week 4)

Before going live:

- ☐ All core features working
 - ☐ Tests passing (>80% coverage)
 - ☐ Performance targets met
 - ☐ Security audit passed
 - ☐ Documentation complete
 - ☐ Deployment rehearsal successful
 - ☐ On-call team trained
 - ☐ Monitoring/alerting set up
 - ☐ Customer communication ready
 - ☐ Incident runbooks prepared
-

📊 METRICS TO TRACK

Daily during development:

Backend:

- API latency (p50, p95, p99)
- Error rate
- Database query time
- Cache hit rate
- Concurrent users

Frontend:

- Bundle size
- Time to interactive
- Lighthouse score
- Component render time

Overall:

- Feature completion %
 - Test coverage %
 - Bug count
 - Deployment frequency
-

🎓 NEXT STEPS

1. **Today:** Complete local setup ✓
 2. **Tomorrow:** Attend kickoff meeting
 3. **This Week:**
 - Submit first PR (small change)
 - Review teammate's PR
 - Contribute to your assigned module
 4. **Week 2:** Advanced modules, testing
 5. **Week 3:** Advanced domains, optimization
 6. **Week 4:** Polish, deploy, launch
-

📖 QUICK REFERENCE

Port Assignments

Service	Port	URL
Frontend	3000	http://localhost:3000
Backend	8000	http://localhost:8000
Database	5432	postgresql://localhost:5432
Redis	6379	redis://localhost:6379
Qdrant	6333	http://localhost:6333
Neo4j	7474	http://localhost:7474

Default Credentials (Dev Only)

Email: dev@shivaai.local
Password: dev-password-change-in-prod

File Structure

```
shivaai-jarvis/  
├─ apps/
```

```
|   |— web/                # React frontend
|   |— desktop/           # Tauri (optional)
|— services/
|   |— gateway/           # FastAPI main app
|   |— agent/             # LangGraph orchestrator
|   |— memory/            # Vector/graph memory
|   |— rag/               # Knowledge ingestion
|   |— voice/             # STT/TTS
|   |— ...
|— infra/                 # K8s, Terraform, Docker
|— scripts/               # Automation
|— tests/                 # Test suites
|— docs/                  # Documentation
|— README.md
```

WELCOME TO THE TEAM!

You're helping build the future of AI. Questions? Blockers? Let us know in #help.

Let's ship something great in 28 days. 

Last updated: 2026-05-30

Next update: 2026-06-06 (weekly review)

Questions? Create an issue or ask in Slack.